

Parcial De Estructuras de Datos No lineales

Estudiante:

Yeimy Estefanía Beltrán Sandoval

Docente:

Javier Alberto Ochoa Ávila

Universidad Sergio Arboleda

Escuela de Ciencias Exacta e Ingeniería

Ciencias de la Computación e Inteligencia Artificial

Bogotá D.C

13 de Marzo de 2026

TABLA DE CONTENIDOS

Introducción.....	3
Objetivos generales.....	3
Temas.....	4
Árboles N-arios.....	4
Definición formal y características:.....	4
Representación en memoria.....	4
Operaciones fundamentales.....	4
Complejidad temporal y espacial.....	4
Aplicaciones reales.....	5
Análisis comparativo con otras estructuras.....	5
Árboles Trie.....	6
Definición formal y características.....	6
Representación en memoria.....	6
Operaciones fundamentales.....	6
Complejidad temporal y espacial.....	6
Aplicaciones reales.....	6
Análisis comparativo con otras estructuras.....	6
Aplicaciones de árboles (expresión, decisión, Huffman, B y B+).....	7
Árboles de expresión.....	7
Definición formal y características.....	7
Representación en memoria.....	7
Operaciones fundamentales.....	7
Complejidad temporal y espacial.....	8
Aplicaciones reales.....	8
Análisis comparativo con otras estructuras.....	8
Árboles de decisión.....	8
Definición formal y características.....	8
Representación en memoria.....	9
Operaciones fundamentales.....	9
Complejidad temporal y espacial.....	9
Aplicaciones reales.....	9
Análisis comparativo con otras estructuras.....	9
Árboles Huffman.....	9
Definición formal y características.....	9
Representación en memoria.....	10
Operaciones fundamentales.....	10
Complejidad temporal y espacial.....	10
Aplicaciones reales.....	10
Análisis comparativo con otras estructuras.....	10
Árboles B.....	11
Definición formal y características.....	11
Representación en memoria.....	11

Operaciones fundamentales.....	11
Complejidad temporal y espacial.....	11
Aplicaciones reales.....	11
Análisis comparativo con otras estructuras.....	11
Árboles B+.....	12
Definición formal y características.....	12
Representación en memoria.....	12
Operaciones fundamentales.....	12
Complejidad temporal y espacial.....	12
Aplicaciones reales.....	13
Análisis comparativo con otras estructuras.....	13
Ejercicios Complejos – Árboles N-arios.....	13
Ejercicio 1: Estructura Organizacional.....	13
Ejercicio 2: Sistema de Archivos.....	14
Ejercicio 3: Árbol Genealógico.....	15
Ejercicio 4: Menú de Aplicación.....	16
Ejercicio 5: Dependencias de Software.....	17
Ejercicios Complejos – Árboles Trie.....	18
Ejercicio 6: Autocompletado.....	18
Ejercicio 7: Corrector Ortográfico.....	19
Ejercicio 8: Clasificador de Intenciones.....	20
Ejercicio 9: Diccionario Multilenguaje.....	21
Ejercicio 10: Motor de Búsqueda.....	22
Hash y Heaps.....	24
Tablas Hash.....	24
Definición formal y características.....	24
Representación en memoria.....	24
Operaciones fundamentales.....	24
Complejidad temporal y espacial.....	24
Aplicaciones reales.....	24
Análisis comparativo con otras estructuras.....	24
Heaps (Montículos).....	25
Definición formal y características.....	25
Representación en memoria.....	25
Operaciones fundamentales.....	25
Complejidad temporal y espacial.....	25
Aplicaciones reales.....	26
Análisis comparativo con otras estructuras.....	26
Colas con Prioridad.....	26
Definición formal y características.....	26
Representación en memoria.....	26
Operaciones fundamentales.....	27
Complejidad temporal y espacial.....	27
Aplicaciones reales.....	27

Análisis comparativo con otras estructuras.....	27
Ejercicios Complejos – Tablas Hash.....	28
Ejercicio 11: Registro de Estudiantes.....	28
Ejercicio 12: Comparación Hash vs Trie.....	28
Ejercicios Complejos – Heaps y Colas de Prioridad.....	29
Ejercicio 13: Heap Mínimo.....	29
Ejercicio 14: Planificador de Tareas.....	29
Ejercicio 15: Simulación de Red.....	29
Conclusiones.....	30
Bibliografía.....	31

Introducción

Este documento presenta el parcial académico correspondiente a las Semanas 5 y 6 de la asignatura Estructuras de Datos No Lineales. El parcial integra investigación, sustentación teórica, ejercicios complejos guiados y criterios de evaluación mediante rúbricas detalladas, alineadas con un enfoque por competencias.

Objetivos generales

- ❖ Investigar y sustentar teóricamente estructuras de datos no lineales avanzadas.
- ❖ Diseñar e implementar soluciones algorítmicas complejas.
- ❖ Analizar la eficiencia computacional de cada estructura.
- ❖ Aplicar las estructuras a problemas reales de la informática y la IA.

Temas

Árboles N-arios

Definición formal y características:

Un árbol N-ario es una estructura de datos que se utiliza para organizar información de forma jerárquica. Donde existe un nodo principal llamado raíz, del cual se desprenden otros nodos llamados hijos. A diferencia de los árboles binarios, donde cada nodo puede tener máximo dos hijos, en los árboles N-arios un nodo puede tener varios hijos dependiendo de la necesidad del problema. Esto permite representar relaciones donde un elemento puede estar conectado con muchos otros al mismo tiempo.

Representación en memoria

En la memoria del computador, los árboles N-arios normalmente se representan utilizando nodos que contienen un dato y una lista o conjunto de referencias hacia sus nodos hijos. Cada nodo guarda la información que se desea almacenar y también las direcciones de memoria de los nodos que dependen de él. Gracias a esto, el programa puede recorrer el árbol y acceder a todos los elementos que forman la estructura.

Operaciones fundamentales

Dentro de las operaciones más importantes que se pueden realizar en un árbol N-ario se encuentran la inserción de nuevos nodos, la eliminación de nodos y la búsqueda de información. También es muy común realizar recorridos por el árbol para visitar todos sus nodos, por ejemplo recorriéndolo desde la raíz hacia sus hijos. Estas operaciones permiten modificar y consultar la estructura de manera organizada.

Complejidad temporal y espacial

En cuanto a la complejidad, las operaciones suelen depender de la altura del árbol. En muchos casos, buscar o insertar un elemento puede tomar un tiempo proporcional al número de niveles que tenga la estructura. Esto significa que mientras el árbol sea más grande, el tiempo para recorrerlo también puede aumentar. En términos de memoria, el árbol necesita espacio para almacenar cada nodo y las referencias hacia sus hijos.

Aplicaciones reales

Los árboles N-arios tienen muchas aplicaciones en la vida real. Un ejemplo muy común es la representación de estructuras jerárquicas como los sistemas de archivos de un computador, donde existen carpetas que contienen otras carpetas y archivos. También se utilizan para representar estructuras organizacionales de empresas, menús de aplicaciones o incluso árboles genealógicos.

Análisis comparativo con otras estructuras

Estructura	Forma de organización	Ventajas	Desventajas
Árbol N-ario	Estructura jerárquica donde cada nodo puede tener varios hijos	Permite representar relaciones jerárquicas complejas de forma natural	Puede consumir más memoria si tiene muchos nodos
Árbol binario	Cada nodo tiene máximo dos hijos	Más fácil de implementar y recorrer	Menos flexible para estructuras con muchos elementos dependientes
Listas	Elementos organizados de forma lineal	Muy simple de implementar	No permite representar jerarquías
Arreglos	Elementos almacenados en posiciones consecutivas	Acceso rápido por índice	Tamaño fijo y poca flexibilidad para estructuras complejas

Árboles Trie

Definición formal y características

El Trie es una estructura de datos especializada que se utiliza principalmente para almacenar y buscar palabras o cadenas de texto. Su función principal se basa en la idea de descomponer cada palabra en caracteres y almacenarlos en forma de árbol. De esta manera, las palabras que comparten un mismo prefijo también comparten parte de la estructura, lo que permite ahorrar espacio y realizar búsquedas de forma eficiente.

Representación en memoria

En memoria, un Trie se representa como un conjunto de nodos conectados entre sí. Cada nodo representa un carácter y contiene referencias hacia otros nodos que representan los siguientes caracteres posibles. Además, algunos nodos tienen una marca especial que indica que en ese punto termina una palabra completa. Esto permite diferenciar entre prefijos y palabras completas.

Operaciones fundamentales

Entre las operaciones más importantes que se realizan en un Trie están la inserción de palabras, la búsqueda de palabras y la verificación de prefijos. Cuando se inserta una palabra, el algoritmo va recorriendo el árbol carácter por carácter y creando nuevos nodos si es necesario. Para buscar una palabra se sigue el mismo recorrido, verificando que todos los caracteres existan dentro de la estructura.

Complejidad temporal y espacial

La complejidad de las operaciones en un Trie depende principalmente de la longitud de la palabra que se está procesando. Esto significa que insertar o buscar una palabra toma un tiempo proporcional al número de caracteres que tenga. En términos de memoria, el Trie puede utilizar más espacio que otras estructuras debido a la gran cantidad de nodos que pueden generarse para almacenar todas las combinaciones de caracteres.

Aplicaciones reales

Los Trie se utilizan en aplicaciones donde es necesario trabajar con grandes conjuntos de palabras. Por ejemplo, se usan en correctores ortográficos, sistemas de autocompletado en buscadores y aplicaciones móviles, así como en motores de búsqueda que necesitan encontrar palabras rápidamente dentro de un diccionario.

Análisis comparativo con otras estructuras

Estructura	Tipo de datos que maneja	Ventajas	Desventajas
Trie	Palabras o cadenas de texto	Búsqueda rápida de palabras y prefijos	Puede ocupar bastante memoria
Lista	Cualquier tipo de dato	Fácil de implementar	Búsqueda lenta si hay muchos elementos
Árbol binario de búsqueda	Datos ordenables	Permite búsquedas eficientes	No está optimizado para prefijos
Tabla hash	Claves y valores	Búsqueda muy rápida	No permite búsquedas por prefijos

Aplicaciones de árboles (expresión, decisión, Huffman, B y B+)

Los árboles son una de las estructuras de datos más utilizadas en informática debido a su capacidad para representar relaciones jerárquicas entre diferentes elementos. Gracias a esta característica, pueden aplicarse en múltiples contextos dentro del desarrollo de software, la inteligencia artificial y el análisis de información. Entre algunas de las aplicaciones más importantes se encuentran los árboles de expresión, los árboles de decisión, el árbol de Huffman y los árboles B y B+.

Árboles de expresión

Definición formal y características

Los árboles de expresión son una estructura de datos que se utiliza para representar expresiones matemáticas o lógicas de forma jerárquica. En este tipo de árbol, cada nodo interno representa un operador, como suma, resta, multiplicación o división, mientras que los nodos hoja representan los operandos, es decir, los números o variables que participan en la operación, una de las características principales es que respetan la precedencia de los operadores, las operaciones que deben realizarse primero aparecen en niveles más profundos del árbol, mientras que las operaciones principales se ubican cerca de la raíz. Esta forma de organización permite visualizar de manera clara el orden en el que deben ejecutarse las operaciones dentro de una expresión.

Representación en memoria

En cuanto a su representación en memoria, los árboles de expresión generalmente se implementan mediante nodos enlazados. Cada nodo contiene un dato que puede ser un operador o un operando, además de referencias a sus nodos hijos. En muchos casos se utilizan árboles binarios, ya que la mayoría de los operadores matemáticos trabajan con dos operandos. De esta forma, cada nodo puede tener un hijo izquierdo y un hijo derecho, lo que permite organizar la expresión de manera clara dentro de la memoria del sistema.

Operaciones fundamentales

Las operaciones fundamentales que se realizan sobre los árboles de expresión incluyen principalmente la construcción del árbol a partir de una expresión, la evaluación del resultado y los recorridos del árbol. Los recorridos, como el recorrido en preorden, inorden y postorden, permiten obtener diferentes representaciones de la expresión, como notación prefija, infija o posfija. Estas representaciones son utilizadas frecuentemente por los compiladores y calculadoras para procesar operaciones matemáticas.

Complejidad temporal y espacial

Respecto a la complejidad, la evaluación de un árbol de expresión suele tener una complejidad temporal de $O(n)$, donde n representa el número de nodos del árbol. Esto se debe a que cada nodo debe visitarse al menos una vez para calcular el resultado de la expresión completa. En términos de espacio, la complejidad también depende del número de nodos que se utilicen para representar la expresión, ya que cada operador y operando ocupa un nodo dentro de la estructura.

Aplicaciones reales

Los árboles de expresión tienen aplicaciones reales principalmente en el desarrollo de compiladores, intérpretes de lenguajes de programación y calculadoras científicas. Estas herramientas utilizan árboles de expresión para analizar y evaluar operaciones escritas por los usuarios. También se utilizan en sistemas de procesamiento simbólico y en algunos motores de bases de datos que deben interpretar expresiones dentro de consultas.

Análisis comparativo con otras estructuras

Estructura	Diferencia principal	Ventaja
Árbol de expresión	Representa operaciones matemáticas de forma jerárquica	Permite evaluar expresiones complejas fácilmente
Árbol binario tradicional	Solo organiza datos sin representar operaciones	Más simple de implementar
Listas o pilas	Guardan datos de forma lineal	Útiles para operaciones simples pero menos visuales

Árboles de decisión

Definición formal y características

Un árbol de decisión es un esquema jerárquico que organiza la toma de decisiones mediante una serie de preguntas y respuestas. Funciona como un mapa donde cada nodo plantea una condición, las ramas muestran las posibles respuestas y los "nodos hoja" entregan el veredicto final. Su gran valor reside en que transforma problemas complejos con múltiples variables en un recorrido visual y estructurado, permitiendo analizar cada combinación de factores que nos conduce a un resultado específico de manera clara y lógica.

Representación en memoria

En cuanto a su representación en memoria, los árboles de decisión suelen implementarse mediante nodos enlazados que contienen una condición o pregunta, además de referencias a los nodos hijos que representan las posibles respuestas. Dependiendo del problema, un nodo puede tener dos o más hijos, lo que permite representar diferentes escenarios dentro del proceso de decisión.

Operaciones fundamentales

Las operaciones fundamentales en los árboles de decisión incluyen la construcción del árbol a partir de un conjunto de datos o reglas, la evaluación de las condiciones para llegar a un resultado y el recorrido del árbol para analizar las diferentes decisiones posibles. Estas

operaciones permiten utilizar el árbol como una herramienta para clasificar información o para apoyar procesos de toma de decisiones.

Complejidad temporal y espacial

Respecto a la complejidad, la evaluación de un árbol de decisión depende principalmente de su profundidad. En muchos casos, el tiempo necesario para llegar a una decisión es proporcional al número de niveles que tiene el árbol. En términos de espacio, la memoria utilizada depende de la cantidad de nodos que se necesiten para representar todas las posibles decisiones.

Aplicaciones reales

Los árboles de decisión tienen muchas aplicaciones en la vida real. Se utilizan en sistemas de inteligencia artificial, análisis de datos, aprendizaje automático y sistemas de recomendación. También se emplean en áreas como el diagnóstico médico, la evaluación de riesgos financieros y la clasificación de información en grandes bases de datos.

Análisis comparativo con otras estructuras

Estructura	Diferencia principal	Ventaja
Árbol de decisión	Representa procesos de toma de decisiones	Facilita analizar diferentes escenarios
Árbol binario	Organiza datos jerárquicamente	Más simple de implementar
Tablas de decisión	Representan reglas en forma tabular	Útiles cuando hay pocas condiciones

Árboles Huffman

Definición formal y características

El árbol de Huffman es una herramienta clave para comprimir información de manera inteligente y sin perder ni un solo dato. Consiste en asignar códigos binarios más cortos a los elementos que más se repiten y dejar los más largos para los que aparecen poco, logrando que el archivo final sea mucho más liviano y eficiente. Para armarlo, se agrupan primero los elementos menos frecuentes hasta formar una estructura única donde cada "hoja" es un símbolo; así, el camino que recorres desde la raíz hasta esa hoja es el que define su código específico.

Representación en memoria

En términos de representación en memoria, el árbol de Huffman se implementa generalmente como un árbol binario. Cada nodo puede tener dos hijos, que representan las posibles decisiones dentro del proceso de codificación. Los nodos internos no contienen

símbolos, sino que representan combinaciones de frecuencias, mientras que las hojas almacenan los caracteres originales.

Operaciones fundamentales

Las operaciones fundamentales incluyen la construcción del árbol a partir de una tabla de frecuencias, la generación de los códigos binarios para cada símbolo y la codificación o decodificación de los datos utilizando el árbol. Estas operaciones permiten transformar información en una forma comprimida y luego reconstruirla cuando sea necesario.

Complejidad temporal y espacial

En cuanto a la complejidad, la construcción del árbol suele tener una complejidad aproximada de $O(n \log n)$, especialmente cuando se utilizan estructuras como colas de prioridad para organizar las frecuencias. En términos de espacio, el árbol requiere memoria proporcional al número de símbolos que se desean codificar.

Aplicaciones reales

El árbol de Huffman tiene aplicaciones importantes en la compresión de archivos y en la transmisión de datos. Se utiliza en diferentes sistemas de compresión y en algunos formatos de almacenamiento digital, ya que permite reducir el tamaño de la información sin afectar su contenido original.

Análisis comparativo con otras estructuras

Estructura	Diferencia principal	Ventaja
Árbol de Huffman	Se usa para compresión de datos	Reduce el tamaño de los archivos
Árbol binario	Solo organiza datos	Más general en su uso
Codificación fija	Todos los símbolos tienen mismo tamaño	Más simple pero menos eficiente

Árboles B

Definición formal y características

El árbol B es una estructura diseñada específicamente para manejar volúmenes masivos de datos en sistemas de almacenamiento y bases de datos. A diferencia de los árboles comunes, este permite que cada nodo guarde múltiples valores y tenga varios hijos, lo que hace que el árbol sea más "chapatito" o bajo. Esta menor altura es clave porque acelera muchísimo las búsquedas.

Su gran ventaja es que se mantiene siempre balanceado, logrando que todos los caminos desde el inicio hasta el final midan casi lo mismo. Al tener los valores ordenados dentro de cada nodo, encontrar la información que necesitas es un proceso sumamente eficiente, incluso cuando hablamos de cantidades de datos gigantescas.

Representación en memoria

En la representación en memoria, cada nodo del árbol B contiene una serie de claves ordenadas y referencias a sus nodos hijos. Estas referencias permiten recorrer el árbol para encontrar un valor específico o insertar nuevos datos manteniendo siempre el orden dentro de la estructura.

Operaciones fundamentales

Las operaciones fundamentales en un árbol B incluyen la búsqueda de datos, la inserción de nuevas claves y la eliminación de elementos. Durante estas operaciones, el árbol puede dividir nodos o redistribuir claves para mantener su estructura balanceada y garantizar que el rendimiento se mantenga eficiente.

Complejidad temporal y espacial

En cuanto a la complejidad, las operaciones de búsqueda, inserción y eliminación generalmente tienen una complejidad de $O(\log n)$. Esto se debe a que la altura del árbol es relativamente pequeña incluso cuando se almacenan grandes cantidades de datos. En términos de espacio, el árbol utiliza memoria proporcional al número de claves almacenadas.

Aplicaciones reales

Los árboles B se utilizan ampliamente en sistemas de gestión de bases de datos, sistemas de archivos y aplicaciones que necesitan manejar grandes volúmenes de información de manera eficiente. Gracias a su estructura, permiten acceder a los datos rápidamente incluso cuando se trabaja con millones de registros.

Análisis comparativo con otras estructuras

Estructura	Diferencia principal	Ventaja
Árbol B	Permite múltiples claves por nodo	Muy eficiente para grandes datos
Árbol binario de búsqueda	Solo permite dos hijos por nodo	Más simple de implementar
Listas enlazadas	Estructura lineal	Más fáciles de recorrer

Árboles B+

Definición formal y características

El árbol B+ es una evolución del árbol B diseñada específicamente para perfeccionar el manejo de bases de datos. Su ventaja es que, a diferencia de otros, guarda toda la información real exclusivamente en los nodos hoja, dejando los nodos internos solo como guías o "letreros" para dirigir las búsquedas de forma ultra rápida.

Lo que lo hace realmente especial es que sus hojas están conectadas entre sí como si fueran una cadena. Esto permite que, si necesitas leer muchos datos seguidos, no tengas que subir y bajar por todo el árbol, sino que simplemente saltas de una hoja a la siguiente en orden. Es la estructura reina cuando se trata de hacer consultas masivas y eficientes en sistemas de almacenamiento modernos.

Representación en memoria

En la representación en memoria, los nodos internos contienen claves que actúan como guías para encontrar la ubicación de los datos, mientras que los nodos hoja contienen los valores reales almacenados. Esta organización mejora el rendimiento en las operaciones de lectura, especialmente cuando se manejan grandes bases de datos.

Operaciones fundamentales

Las operaciones fundamentales en los árboles B+ incluyen la búsqueda de registros, la inserción de nuevos datos y la eliminación de información. Al igual que en el árbol B, estas operaciones pueden provocar divisiones o reorganizaciones de nodos para mantener el árbol balanceado y garantizar un acceso eficiente a los datos.

Complejidad temporal y espacial

En cuanto a la complejidad, las operaciones principales también suelen tener una complejidad de $O(\log n)$, ya que la estructura del árbol se mantiene balanceada. Además, gracias a los enlaces entre las hojas, los recorridos secuenciales pueden realizarse de forma muy rápida.

Aplicaciones reales

El árbol B+ se utiliza ampliamente en los sistemas de gestión de bases de datos y en los índices de almacenamiento de información. Muchos motores de bases de datos modernos utilizan esta estructura para organizar sus índices, ya que permite realizar búsquedas rápidas y acceder a grandes cantidades de datos de manera eficiente.

Análisis comparativo con otras estructuras

Estructura	Diferencia principal	Ventaja
------------	----------------------	---------

Árbol B+	Los datos se guardan solo en las hojas	Mejor para consultas grandes
Árbol B	Los datos se guardan en todos los nodos	Estructura más simple
Árbol binario	Solo tiene dos hijos por nodo	Fácil de implementar

Ejercicios Complejos – Árboles N-arios

Ejercicio 1: Estructura Organizacional

Problema: Modelar la jerarquía de una universidad mediante un árbol N-ario.

Pasos:

1. Identificar la raíz (Rectoría).
2. Definir nodos intermedios (Facultades, Programas).
3. Definir nodos hoja (Asignaturas).
4. Implementar inserción dinámica.
5. Implementar recorrido por niveles.
6. Calcular altura y profundidad.

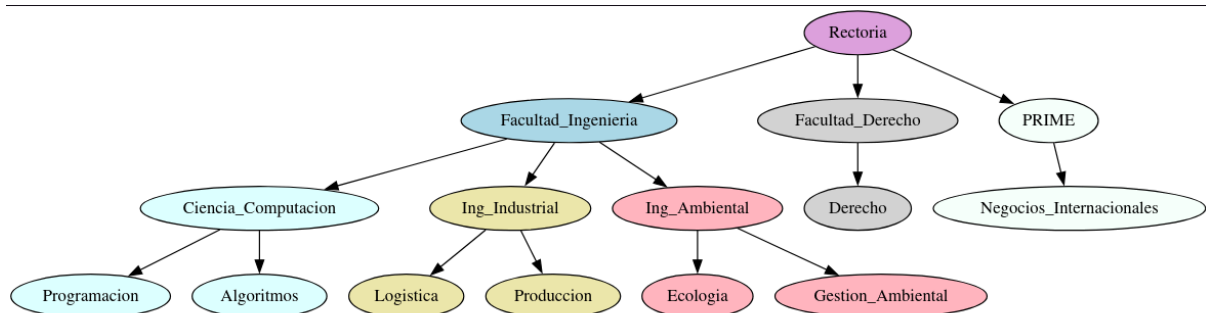


Imagen 1 . Árbol N-ario ejercicio 1

En este ejercicio se implementa un árbol N-ario para representar la estructura organizacional de una universidad. La raíz del árbol corresponde a la rectoría, mientras que los nodos intermedios representan las facultades o escuelas. Finalmente, los nodos hoja corresponden a los programas académicos que ofrece cada facultad.

En el ejemplo se construye un árbol donde la rectoría es el nodo principal. A partir de este nodo se agregan tres facultades: la Facultad de Ingeniería, la Escuela Mayor de Derecho y PRIME. Cada una de estas facultades contiene programas académicos específicos.

Complejidad

- Inserción de nodos: $O(1)$
- Recorrido del árbol: $O(n)$

Ejercicio 2: Sistema de Archivos

Simular un sistema de carpetas y archivos.

Pasos:

1. Representar carpetas como nodos internos.
2. Representar archivos como hojas.
3. Implementar búsqueda por nombre.
4. Implementar eliminación recursiva.
5. Analizar complejidad.

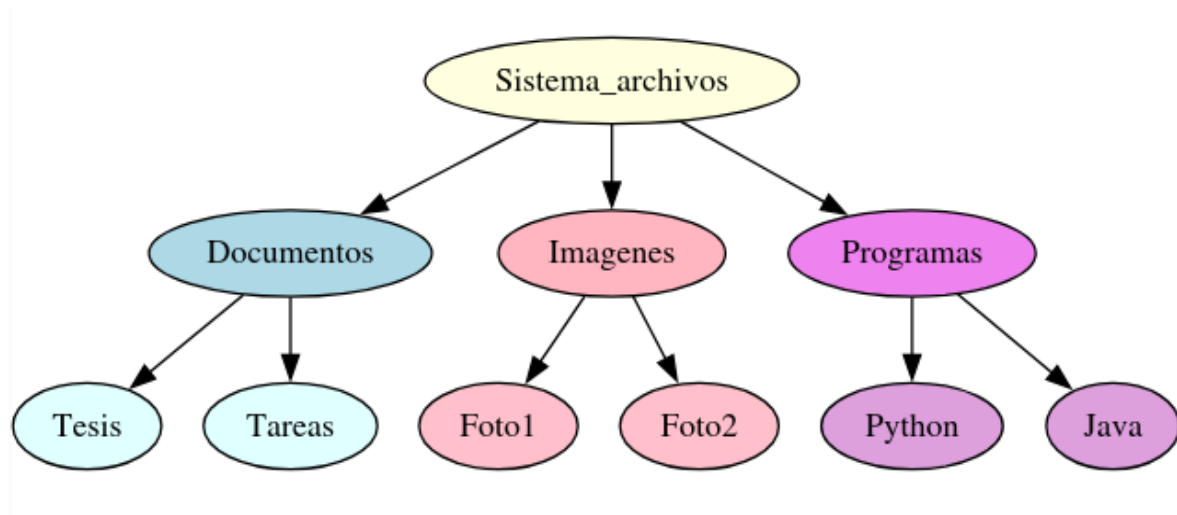


Imagen 2 . Árbol N-ario ejercicio 2

Este ejercicio representa la organización de archivos y carpetas mediante una estructura de árbol. Cada nodo puede representar una carpeta o un archivo, y cada carpeta puede contener otros elementos dentro de ella.

Se construye un sistema de archivos simple donde existe una carpeta raíz llamada "Sistema_archivos". Dentro de esta carpeta se agregan dos subcarpetas llamadas "documentos" e "imagenes", y cada una contiene diferentes archivos.

Complejidad

- Inserción: $O(1)$
- Recorrido: $O(n)$

Ejercicio 3: Árbol Genealógico

1. Definir estructura del nodo.
2. Insertar relaciones padre-hijo.
3. Buscar ancestros comunes.
4. Calcular generaciones.
5. Analizar grado del árbol.

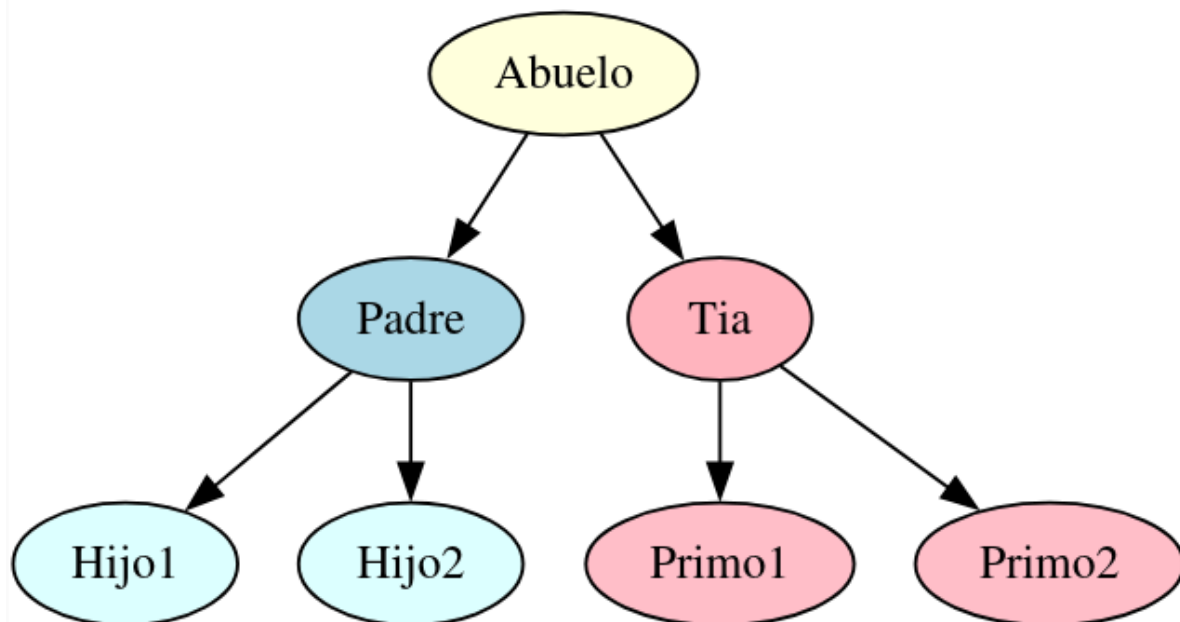


Imagen 3 . Árbol N-ario ejercicio 3

Un árbol genealógico permite representar las relaciones familiares entre diferentes generaciones. Cada nodo corresponde a una persona y los nodos hijos representan sus descendientes.

Explicación del ejemplo

En el ejemplo se crea un árbol donde el nodo raíz representa al abuelo. A partir de él se agregan sus hijos (padre y tía), y posteriormente se agregan los hijos de cada uno.

Complejidad

- Inserción de hijos: $O(1)$
- Recorrido: $O(n)$

Ejercicio 4: Menú de Aplicación

1. Identificar opciones y subopciones.
2. Construir árbol.
3. Simular navegación.
4. Contar nodos por nivel.
5. Evaluar eficiencia.

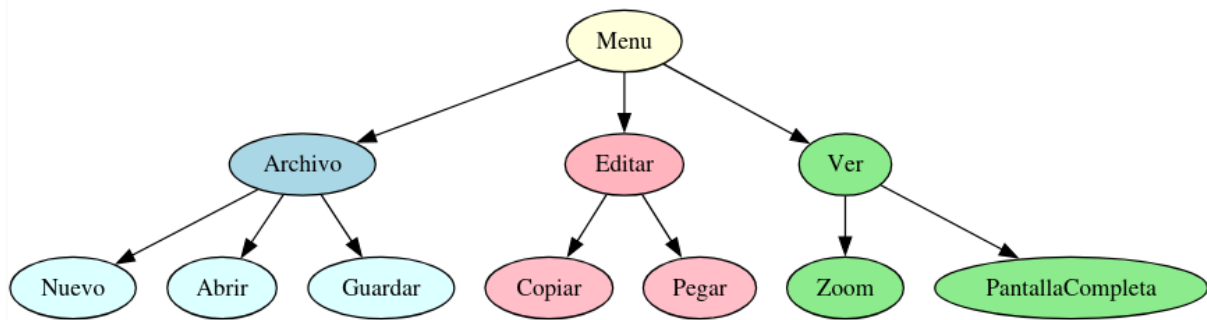


Imagen 4. Árbol N-ario ejercicio 4

Los menús de muchas aplicaciones están organizados de forma jerárquica. Esto permite representar las opciones principales y sus subopciones mediante una estructura de árbol.

Se crea un menú principal que contiene dos opciones: "Archivo" y "Editar". Cada una de estas opciones tiene subopciones como "Nuevo", "Guardar", "Copiar" y "Pegar".

Complejidad

- Inserción: $O(1)$

Ejercicio 5: Dependencias de Software

1. Identificar módulos.
2. Definir dependencias.
3. Construir árbol.
4. Analizar impacto de cambios.
5. Evaluar reutilización.

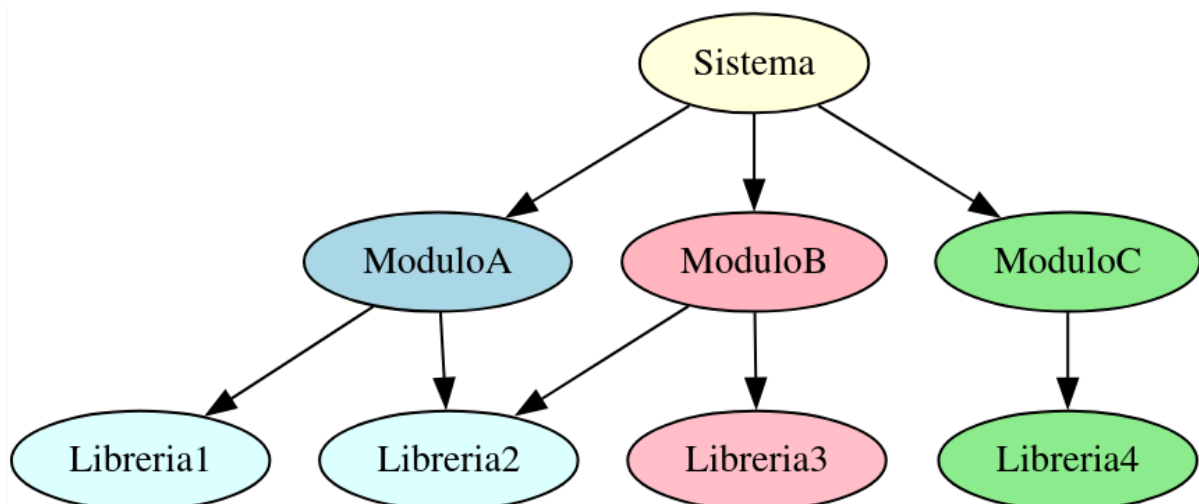


Imagen 5. Árbol N-ario ejercicio 5

En el desarrollo de software, los programas suelen depender de diferentes módulos o librerías. Estas dependencias pueden representarse mediante un árbol.

Se crea un sistema principal que depende de dos módulos llamados "ModuloA" y "ModuloB". Cada módulo tiene dependencias adicionales como librerías externas.

Complejidad

- Inserción: $O(1)$

Ejercicios Complejos – Árboles Trie

Ejercicio 6: Autocompletado

1. Insertar palabras.
2. Marcar finales de palabra.
3. Buscar prefijos.
4. Generar sugerencias.
5. Analizar complejidad $O(L)$.

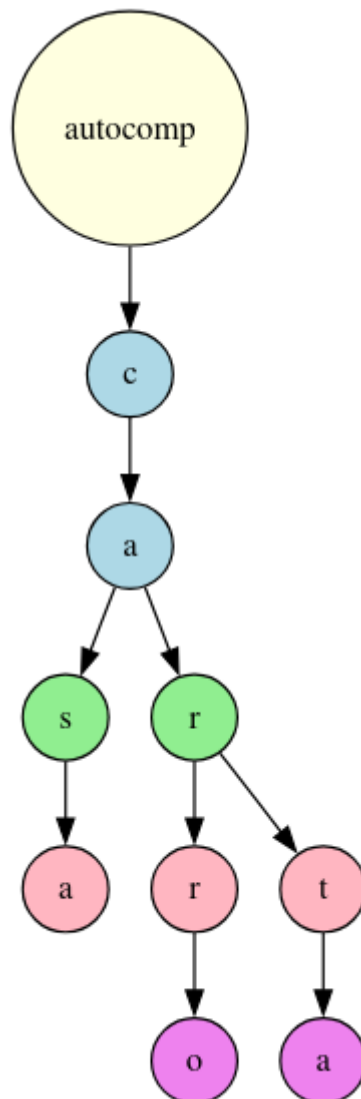


Imagen 6. Árbol Trie ejercicio 6

El Trie es una estructura de datos utilizada para almacenar palabras de manera eficiente. Cada nodo representa un carácter de una palabra.

Se insertan varias palabras como "casa", "carro" y "carta". Luego se realiza una búsqueda para verificar si las palabras existen dentro del Trie.

Complejidad

- Inserción: $O(n)$
- Búsqueda: $O(n)$

(donde n es la longitud de la palabra)

Ejercicio 7: Corrector Ortográfico

1. Cargar diccionario.
2. Detectar palabras inválidas.
3. Generar sugerencias.
4. Comparar con Hash.
5. Analizar memoria.

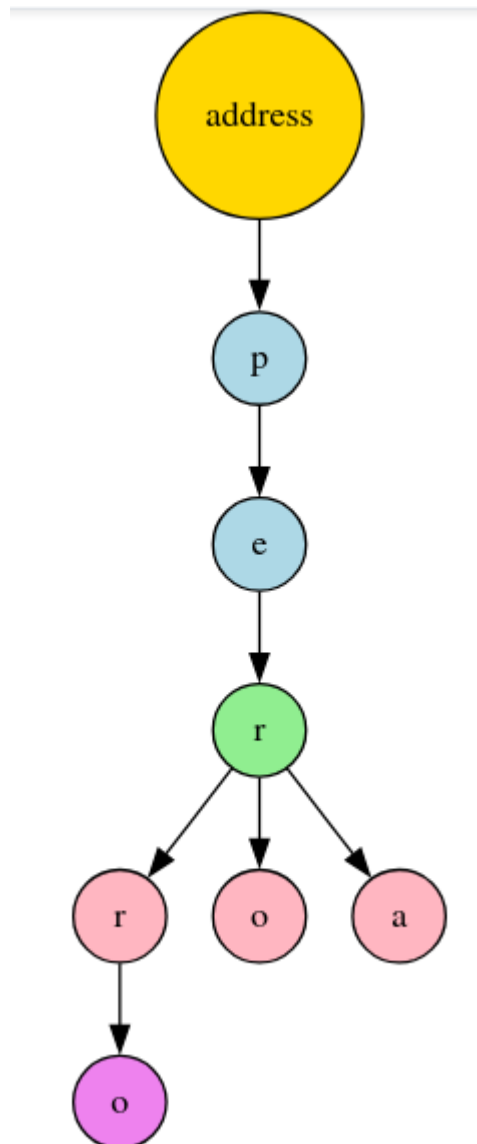


Imagen 7. Árbol Trie ejercicio 7

Un corrector ortográfico puede utilizar un Trie para verificar si una palabra existe dentro de un diccionario.

Se cargan palabras como "hola", "mundo" y "python" dentro del sistema. Luego se verifica si ciertas palabras están presentes en el diccionario.

Complejidad

- Verificación: $O(n)$

Ejercicio 8: Clasificador de Intenciones

1. Agrupar palabras por prefijo.
2. Asociar intenciones.
3. Simular respuesta.

4. Evaluar precisión.
5. Escalar estructura.

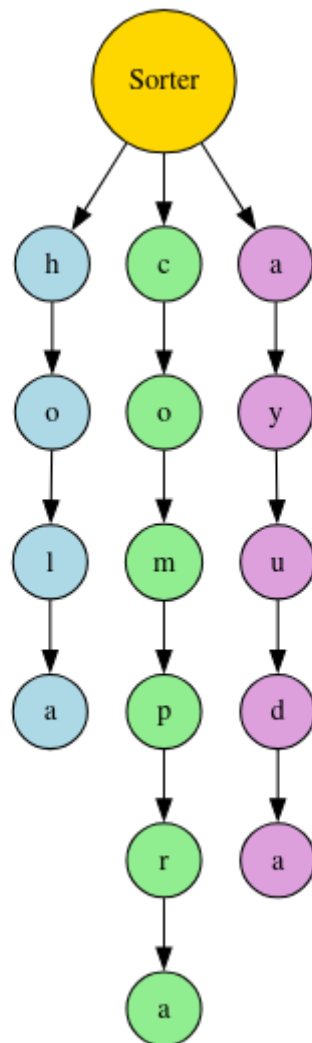


Imagen 8. Árbol Trie ejercicio 8

Los árboles de decisión son utilizados en inteligencia artificial y sistemas expertos para representar decisiones basadas en preguntas.

Explicación del ejemplo

Se crea un árbol simple con la pregunta "¿Está lloviendo?". Dependiendo de la respuesta, se toma una acción diferente.

Complejidad

- Evaluación: $O(n)$

Ejercicio 9: Diccionario Multilenguaje

1. Insertar palabras por idioma.

2. Asociar significados.
3. Buscar por prefijo.
4. Optimizar memoria.
5. Analizar complejidad.

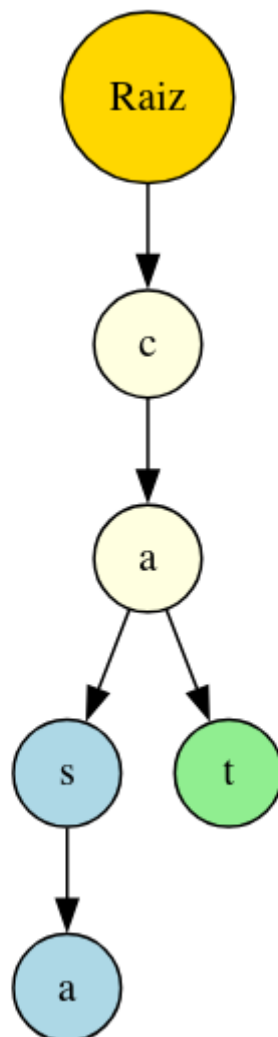


Imagen 9. Árbol Trie ejercicio 9

Un diccionario permite almacenar palabras junto con sus significados para facilitar la búsqueda de información.

Se agregan palabras como "casa" y "cat", cada una con su respectivo significado. Luego se realiza una búsqueda para encontrar el significado de una palabra.

Complejidad

- Búsqueda: $O(n)$

Ejercicio 10: Motor de Búsqueda

1. Insertar títulos académicos.

2. Autocompletar consultas.
3. Priorizar resultados.
4. Integrar heap.
5. Evaluar rendimiento.

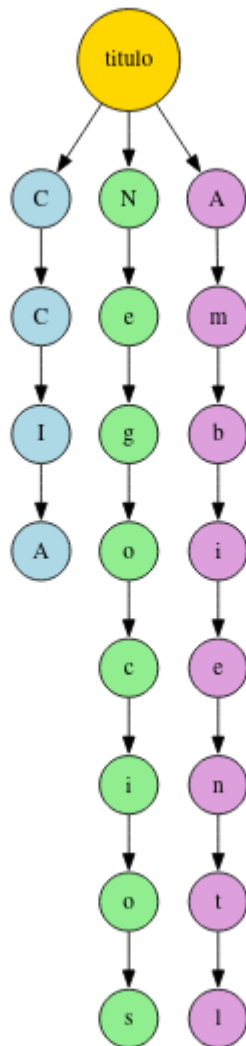


Imagen 10. Árbol Trie ejercicio 10

Un motor de búsqueda puede organizar términos relacionados mediante estructuras de datos que permitan asociar diferentes conceptos.

Se agregan términos como "CCIA" y "negocios", y se establece una relación entre ellos como títulos de una universidad la cual los relaciona.

Complejidad

- Inserción: $O(1)$

Hash y Heaps

Tablas Hash

Definición formal y características

La tabla hash es una estructura de datos que permite almacenar información de manera muy eficiente utilizando una función llamada función hash. Esta función transforma una clave o dato en una posición específica dentro de una tabla. Gracias a esto, es posible acceder a la información de manera muy rápida sin necesidad de recorrer todos los elementos almacenados.

Representación en memoria

En memoria, una tabla hash se representa como un arreglo o conjunto de posiciones donde se guardan los datos. Cada elemento se coloca en una posición determinada según el resultado de la función hash. Sin embargo, puede ocurrir que dos datos diferentes produzcan la misma posición, lo que se conoce como colisión. Para resolver este problema se utilizan diferentes técnicas como listas enlazadas o direccionamiento abierto.

Operaciones fundamentales

Las operaciones principales en una tabla hash son la inserción de datos, la búsqueda y la eliminación. Cuando se inserta un elemento, se calcula su valor hash para determinar en qué posición de la tabla se debe almacenar. Para buscar un elemento se realiza el mismo proceso, lo que permite encontrarlo rápidamente si la función hash está bien diseñada.

Complejidad temporal y espacial

En términos de complejidad, las tablas hash son conocidas por tener tiempos de acceso muy rápidos. En condiciones ideales, buscar o insertar un elemento puede tomar tiempo constante, lo que significa que el tiempo no depende del número de elementos almacenados. Sin embargo, cuando existen muchas colisiones, el rendimiento puede disminuir.

Aplicaciones reales

Las tablas hash tienen muchas aplicaciones en sistemas informáticos. Por ejemplo, se utilizan en bases de datos, en la implementación de diccionarios o mapas en lenguajes de programación, en sistemas de autenticación y en el almacenamiento de información donde se necesita acceder rápidamente a los datos.

Análisis comparativo con otras estructuras

Estructura	Forma de acceso	Ventajas	Desventajas
------------	-----------------	----------	-------------

Tabla Hash	Mediante una función hash	Acceso muy rápido a los datos	Puede haber colisiones
Árbol de búsqueda	Comparación entre nodos	Mantiene los datos ordenados	Búsqueda más lenta que hash
Lista enlazada	Recorrido secuencial	Fácil de implementar	Búsqueda lenta
Arreglo	Acceso por índice	Muy rápido cuando se conoce la posición	Poco flexible para datos dinámicos

Heaps (Montículos)

Definición formal y características

El heap es una estructura de datos que se utiliza principalmente para manejar conjuntos de elementos donde siempre se necesita acceder rápidamente al valor mayor o menor. Se basa en un árbol binario casi completo en el cual los nodos cumplen una propiedad específica: en un max-heap el valor del nodo padre es mayor que el de sus hijos, mientras que en un min-heap ocurre lo contrario.

Representación en memoria

En la memoria del computador, el heap generalmente se representa utilizando un arreglo. Aunque conceptualmente se trata de un árbol, almacenar los elementos en un arreglo permite calcular fácilmente la posición de los hijos y del padre mediante simples operaciones matemáticas. Esto hace que la implementación sea más sencilla y eficiente.

Operaciones fundamentales

Las operaciones fundamentales en un heap incluyen la inserción de elementos, la eliminación del elemento principal y la reorganización de la estructura para mantener su propiedad. Cuando se inserta un nuevo elemento, este se coloca inicialmente al final del arreglo y luego se ajusta su posición para mantener el orden del heap. De forma similar, cuando se elimina el elemento principal, se reorganiza el árbol para conservar la estructura.

Complejidad temporal y espacial

La complejidad de estas operaciones suele ser logarítmica con respecto al número de elementos. Esto significa que incluso cuando la estructura crece, las operaciones siguen siendo relativamente rápidas. En términos de espacio, el heap requiere memoria para almacenar todos los elementos dentro del arreglo.

Aplicaciones reales

Los heaps se utilizan en muchos algoritmos importantes. Uno de los ejemplos más conocidos es el algoritmo de ordenamiento llamado HeapSort. También se utilizan en estructuras como las colas de prioridad, que permiten procesar primero los elementos más importantes dentro de un conjunto de datos.

Análisis comparativo con otras estructuras

Estructura	Propósito principal	Ventajas	Desventajas
Heap	Obtener rápidamente el mayor o menor elemento	Muy eficiente para colas de prioridad	No es eficiente para buscar un elemento específico
Árbol binario	Organización jerárquica de datos	Permite búsquedas estructuradas	No siempre mantiene prioridad
Lista	Secuencia de elementos	Implementación sencilla	Ordenar puede ser costoso
Arreglo	Almacenamiento simple	Acceso directo por índice	No mantiene automáticamente un orden de prioridad

Colas con Prioridad

Definición formal y características

Las colas con prioridad son estructuras de datos que permiten almacenar elementos junto con un nivel de prioridad asociado. A diferencia de las colas tradicionales, donde los elementos se atienden en el orden de llegada, en las colas con prioridad los elementos se procesan según su importancia; es decir, un elemento con mayor prioridad puede ser atendido antes que otros que hayan llegado primero. Cada elemento tiene un valor que determina su prioridad, y según la implementación, puede salir primero el de mayor o menor prioridad. Estas colas se implementan comúnmente usando heaps, ya que facilitan organizar y extraer los elementos de manera eficiente respetando su nivel de prioridad.

Representación en memoria

En cuanto a la representación en memoria, las colas con prioridad generalmente se implementan mediante arreglos que representan un árbol binario implícito, como ocurre en los heaps. En esta representación, cada posición del arreglo corresponde a un nodo del árbol, y las relaciones entre padres e hijos se determinan mediante operaciones

matemáticas simples. Esto permite almacenar y reorganizar los elementos de manera eficiente cuando se insertan o eliminan.

Operaciones fundamentales

Las operaciones fundamentales de una cola con prioridad incluyen la inserción de un nuevo elemento, la eliminación del elemento con mayor o menor prioridad y la consulta del elemento prioritario sin eliminarlo. Cuando se inserta un nuevo elemento, la estructura se reorganiza para mantener el orden de prioridad. De forma similar, cuando se elimina el elemento prioritario, los demás elementos se ajustan para conservar la propiedad del heap.

Complejidad temporal y espacial

En términos de complejidad, las operaciones de inserción y eliminación suelen tener una complejidad aproximada de $O(\log n)$ cuando se implementan mediante heaps, ya que puede ser necesario reorganizar la estructura para mantener el orden. La operación de consulta del elemento prioritario generalmente tiene una complejidad $O(1)$, ya que este elemento se encuentra en la raíz de la estructura. En cuanto al espacio, la complejidad depende del número de elementos almacenados, por lo que normalmente es $O(n)$.

Aplicaciones reales

Las colas con prioridad tienen muchas aplicaciones en diferentes áreas de la informática. Se utilizan en algoritmos de planificación de tareas en sistemas operativos, donde algunos procesos deben ejecutarse antes que otros según su importancia. También se emplean en algoritmos de grafos como el algoritmo de Dijkstra, que encuentra el camino más corto entre nodos en una red. Además, se utilizan en simulaciones, sistemas de gestión de eventos y aplicaciones donde es necesario procesar primero los elementos más urgentes o importantes.

Análisis comparativo con otras estructuras

Estructura	Forma de funcionamiento	Ventaja
Cola con prioridad	Los elementos se atienden según su prioridad	Permite procesar primero los elementos más importantes
Cola simple	Los elementos se atienden en orden de llegada (FIFO)	Fácil de implementar
Pila	El último elemento insertado es el primero en salir (LIFO)	Útil para operaciones recursivas

Lista	Los elementos se recorren secuencialmente	Muy flexible para almacenar datos
-------	---	-----------------------------------

Ejercicios Complejos – Tablas Hash

Ejercicio 11: Registro de Estudiantes

1. Diseñar función hash.
2. Insertar registros.
3. Manejar colisiones.
4. Implementar eliminación.
5. Analizar factor de carga.

Para este ejercicio se diseñó una clase en Python que permite insertar y buscar registros de estudiantes utilizando su identificador como clave. Cada vez que se inserta un nuevo elemento, la función hash calcula la posición donde se almacenará dentro de la tabla. En caso de que dos claves generen la misma posición, se utiliza una lista en esa posición para almacenar ambos elementos.

En el ejemplo de uso se insertan varios estudiantes con su respectivo ID y nombre, y posteriormente se realiza una búsqueda para verificar si un estudiante específico se encuentra almacenado en la tabla.

Ejercicio 12: Comparación Hash vs Trie

1. Implementar ambas estructuras.
2. Medir tiempos de búsqueda.
3. Comparar memoria.
4. Evaluar escalabilidad.
5. Concluir uso óptimo.

En este ejercicio se creó una clase para representar el nodo del Trie y otra clase para gestionar la estructura completa. Cada nodo puede tener múltiples hijos, cada uno asociado a un carácter diferente.

En el ejemplo se insertan varias palabras dentro del Trie y posteriormente se realiza una búsqueda para verificar si determinadas palabras existen dentro de la estructura.

Los Tries son ampliamente utilizados en aplicaciones como correctores ortográficos, motores de búsqueda y sistemas de autocompletado de texto.

Ejercicios Complejos – Heaps y Colas de Prioridad

Ejercicio 13: Heap Mínimo

1. Insertar elementos.
2. Extraer mínimo.
3. Reorganizar heap.
4. Analizar $O(\log n)$.
5. Probar casos límite.

En este ejercicio se realizó utilizando la librería `heapq` de Python, que permite manejar este tipo de estructuras de manera eficiente. La clase desarrollada permite insertar elementos dentro del heap y extraer el elemento mínimo.

En el ejemplo se insertan varios números en la estructura y posteriormente se extrae el valor mínimo, demostrando cómo el heap organiza automáticamente los elementos para mantener su propiedad.

Ejercicio 14: Planificador de Tareas

1. Definir tareas con prioridad.
2. Insertar en heap.
3. Simular ejecución.
4. Analizar eficiencia.
5. Evaluar justicia del sistema.

En este ejercicio se implementa una cola con prioridad, en lugar de procesar los elementos únicamente en el orden en que llegan, la cola con prioridad siempre procesa primero el elemento que tenga mayor prioridad.

Para la implementación se utilizó nuevamente la librería `heapq`, que permite organizar automáticamente los elementos según su nivel de prioridad.

En el ejemplo se agregan varias tareas con diferentes niveles de prioridad, y posteriormente se ejecuta la tarea con mayor prioridad. Este tipo de estructuras es comúnmente utilizado en sistemas operativos para la planificación de procesos y en sistemas de gestión de tareas.

Ejercicio 15: Simulación de Red

1. Modelar paquetes.
2. Asignar prioridades.
3. Procesar con heap.

4. Medir tiempos.
5. Analizar resultados.

En este ejercicio se realiza una simulación simple del procesamiento de paquetes dentro de una red de comunicación. En este tipo de sistemas, algunos paquetes de información pueden tener mayor prioridad que otros, por lo que deben ser procesados primero.

Para representar este comportamiento se utilizó una **cola con prioridad**, donde cada paquete se inserta con un nivel de prioridad determinado. La estructura organiza automáticamente los paquetes para garantizar que aquellos con mayor prioridad sean procesados primero.

En el ejemplo se envían varios paquetes de información a la red con diferentes prioridades. Posteriormente se procesa el paquete con mayor prioridad, simulando el funcionamiento de un sistema real de transmisión de datos.

Este tipo de estructuras se utiliza en redes de telecomunicaciones, sistemas de transmisión de video y gestión de tráfico en internet.

Conclusión

Durante este trabajo se comprendió la importancia de las estructuras de datos no lineales en informática, ya que permiten organizar la información de forma jerárquica o con relaciones más complejas que las estructuras lineales. Se analizaron diferentes estructuras como árboles N-arios, tries, tablas hash, heaps y colas con prioridad, entendiendo su funcionamiento, operaciones y aplicaciones. Además, mediante su implementación en Python, se pudieron aplicar de forma práctica los conceptos teóricos vistos en clase, como la organización de nodos, funciones hash y manejo de prioridades. En conclusión, el trabajo permitió fortalecer habilidades de programación y comprender la importancia de elegir la estructura de datos adecuada para mejorar la eficiencia en el manejo de la información.

Bibliografía

Cormen, T., Leiserson, C., Rivest, R., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Python*. Wiley.

Downey, A. (2015). *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media.

Python Software Foundation. (2024). *Python Documentation*.